Cmake 与 Makefile 基础性学习

Makefile 相当于由 make 指令直接编译和连接规则，一般来说依赖于 shell 指令，过程式构建，直接描述如何一步步构建内容，仍然以昨天成功的 hello.c 为例，记录如何使用 makefile

1.基础语法

目标：依赖文件 表示目标文件由依赖文件按照以下规则生成

依赖文件可以是多个，用空格隔开

<tab 缩进>要执行的命令，\为续写符，将一条长命令或者长变量拆解成多行来书写

定义编译器和编译选项

CC = gcc

#定义编译选项，-wall 为开启常用警告，-g 保留调试信息，-02 优化代码

CFLAGS = -Wall -g -02

定义目标文件（可执行文件名）

TARGET = hello

定义源文件和对应的目标文件

SRCS =hello.c

默认目标（执行 make 时构建的第一个目标）

all: $(TARGET)

链接目标文件生成最终的可执行文件，即如何从源文件生成可执行文件（$CC 开头必须是一个 tab）

$(TARGET): $(OBJS)

$(CC) $(CFLAGS) -o $(TARGET) $(SRC)

用于清理构建产物

clean:

rm -f $(TARGET)

2.语法元素与内置函数

变量定义与引用

变量名=值 #使用时展开的变量

变量名:=值 #简单展开变量，在定义的时候就展开

变量名？=值#变量未定义则赋值，已定义则忽略这条指令

变量名+=值#追加内容

${变量名},${变量名}#引用该变量的值

预定义变量

CC C 编译器，默认为 cc，指向 gcc

CXX CPP 编译器，默认为 g++

CFLAGS C 编译选项，如-wall -02

CXXFLAGS CPP 编译选项

LDFLAGS 链接器选项，如-L 库路径

LDLIBS 链接的库，比如-lm –lpthread

MAKE make 指令本身，用于递归调用

规则命令（shell 命令，每个命令之前用一个 tab 缩进）

gcc -c main.c -o main.o 将 .c 编译成目标文件 .o（只编译不链接）。

gcc -o program main.o utils.o 将多个 .o 链接成可执行文件。

g++ -std=c++11 -o app main.cpp C++ 编译并链接。

ar rcs libmylib.a file1.o file2.o 创建静态库 .a。

2、文件与目录操作命令

用于创建、删除、复制、移动文件或目录，常见于 clean 目标或准备构建环境。

命令示例 作用

rm -f *.o $(TARGET) 强制删除所有 .o 文件和最终目标文件。

mkdir -p build/bin 递归创建目录（-p 确保父目录不存在时也成功）。

cp config.ini $(BIN_DIR)/ 复制配置文件到输出目录。

mv temp.file final.file 重命名或移动文件。

ln -sf libsomething.so.1 libsomething.so 创建符号链接（常用于库版本管理）。

自动变量（规则命令调用）

$@ 当前规则的 目标文件名。

$< 当前规则的 第一个依赖文件名。

$^ 当前规则的 所有依赖文件列表（去重）。

$? 所有 比目标新的依赖文件列表。

$* 模式规则中，% 匹配的 主干部分（不含后缀）

例子:

Hello : hello.c

gcc –o $@ $^

$@ = hello , $^ = hello.c

常用内置函数

$(wildcard *.c) 获取当前目录下所有 .c 文件的列表。

$(patsubst %.c,%.o,$(SRCS)) 将 SRCS 中所有 .c 替换为 .o。

$(SRCS:.c=.o) 简化的替换写法，功能同上（但只能替换后缀）。

$(notdir $(PATH)) 从路径中提取文件名，去掉目录部分。

$(dir $(PATH)) 提取路径中的目录部分。

$(shell 命令) 执行 shell 命令并返回结果。

$(foreach var, list, text) 循环处理，对 list 中的每个词执行 text（可用 $(var)）。

$(if condition, then, else) 条件判断。

条件判断语法

ifeq (A, B) ... endif 如果 A 等于 B 则执行。

ifneq (A, B) ... endif 如果不相等则执行。

ifdef 变量名 ... endif 如果变量已定义则执行。

ifndef 变量名 ... endif 如果变量未定义则执行。

注意： 条件语句不能写在命令前面，必须写在目标、依赖、命令的外面。

模式规则与静态模式规则

语法 作用解释

%.o: %.c 模式规则：表示任何 .o 文件都依赖于同名的 .c 文件，并按下方命令生成。

$(OBJS): %.o: %.c 静态模式规则：只对 $(OBJS) 列表中的文件应用该模式，作用更精确。

伪目标与特殊目标

目标 作用解释

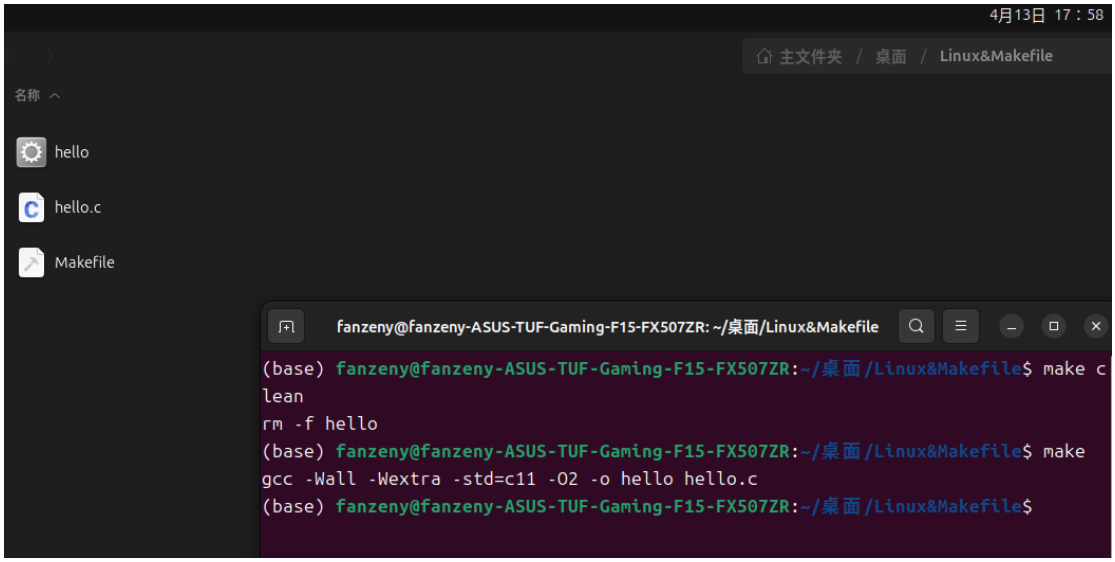
.PHONY: clean 声明 clean 是一个伪目标，即使目录下有名为 clean 的文件，make clean 也会执行命令。

all 惯例目标，通常作为 Makefile 第一个目标，表示构建所有最终产物。

clean 惯例目标，用于删除所有构建产物。

install 惯例目标，用于将编译好的文件安装到系统目录。

实验预期：

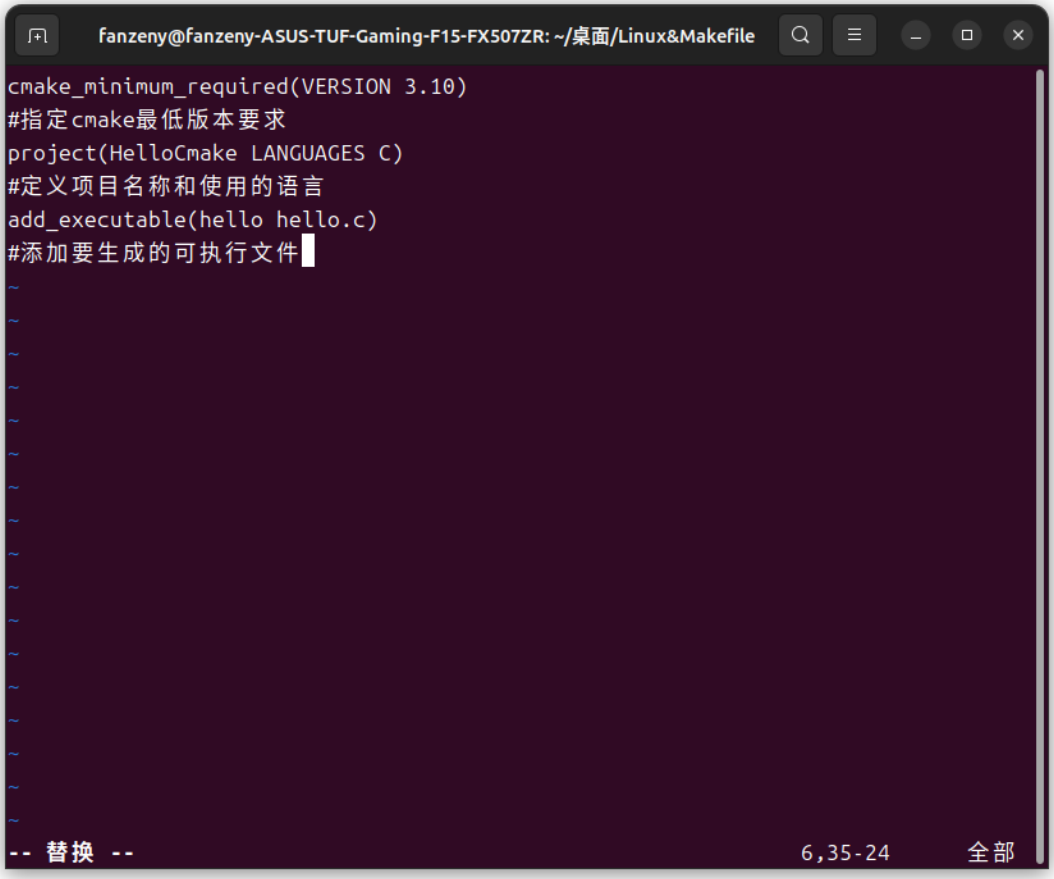


#wall 开启编译警告，wextra 开启额外警告，-std=c11 使用 c11 标准编译，-02 优化级别为 2，-o 输出指定文件名

CmakeList

相比 makefile 更加智能的 make 文件

例子:



配置与编译

Mkdir build #新建一个 build 文件夹

cd build #转到

cmake .. #运行 cmake 配置项目，后边的两个点表示上层目录的 CMakeList

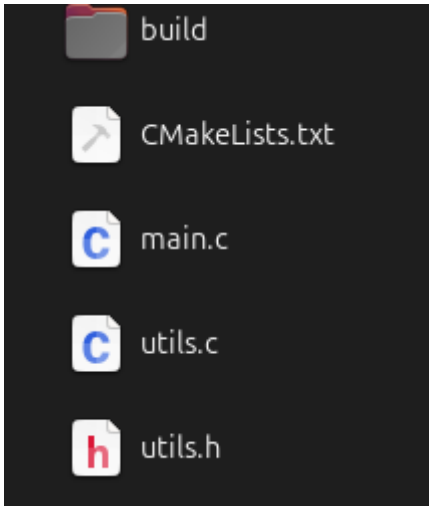
make #编译

实验预期:

```
fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR: ~/桌面/Linux&Makefil...
Specify --help for usage, or press the help button on the CMake GUI.
(base) fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR:~/桌面/Linux&Makefile/build$
cmake ..
-- The C compiler identification is GNU 13.3.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Configuring done (0.3s)
-- Generating done (0.0s)
-- Build files have been written to: /home/fanzeny/桌面/Linux&Makefile/build
(base) fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR:~/桌面/Linux&Makefile/build$
make
[ 50%] Building C object CMakeFiles/hello.dir/hello.c.o
[100%] Linking C executable hello
[100%] Built target hello
(base) fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR:~/桌面/Linux&Makefile/build$
./hello
Hello, World!
(base) fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR:~/桌面/Linux&Makefile/build$
make clean
(base) fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR:~/桌面/Linux&Makefile/build$
```

多源文件自动收集与管理

目录结构为多个.c 与 cmakelist



可以使用 file(GLOB MY_SOURCE “*.c”)

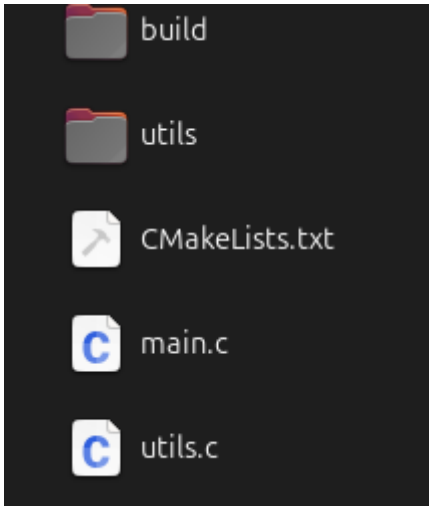
使用${MY_SOURCE}调用，和 makelist 一致

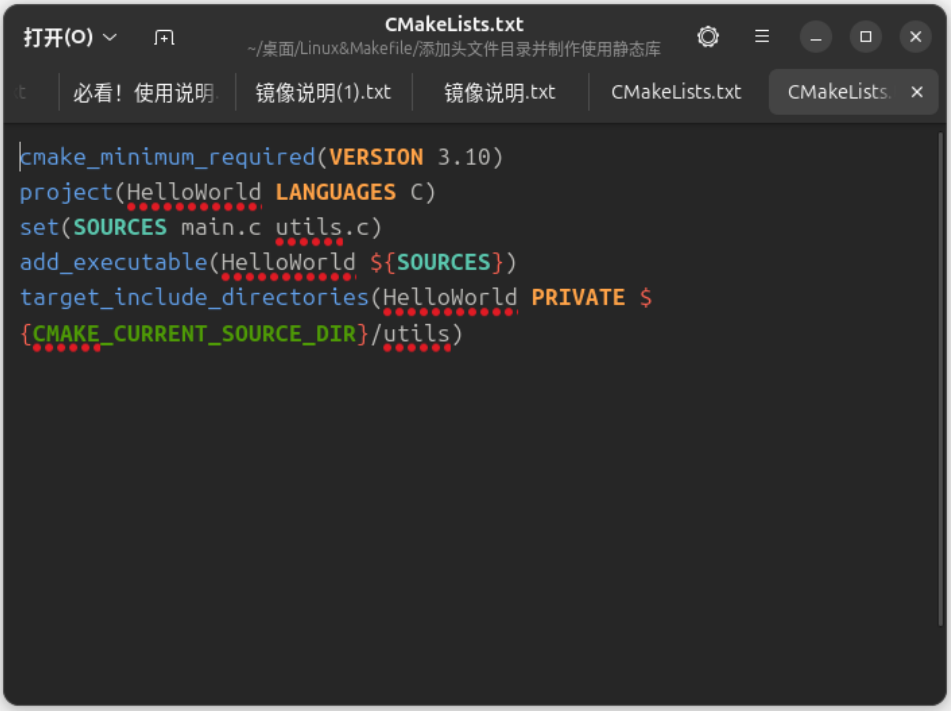
实验预期:

```
fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR: ~/桌面/Linux&Makefil...
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
CMake Error: CMAKE_LANGUAGUES_COMPILER not set, after EnableLanguage
-- Configuring incomplete, errors occurred!
(base) fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR:~/桌面/Linux&Makefile/多源文
件实验/build$ cmake ..
-- Configuring done (0.0s)
-- Generating done (0.0s)
-- Build files have been written to: /home/fanzeny/桌面/Linux&Makefile/多源文件
实验/build
(base) fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR:~/桌面/Linux&Makefile/多源文
件实验/build$ make
[100%] Built target myapp
(base) fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR:~/桌面/Linux&Makefile/多源文
件实验/build$ ./main
bash: ./main: 没有那个文件或目录
(base) fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR:~/桌面/Linux&Makefile/多源文
件实验/build$ ./myapp
Hello, World!
(base) fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR:~/桌面/Linux&Makefile/多源文
件实验/build$
```

添加头文件目录

目录结构如下



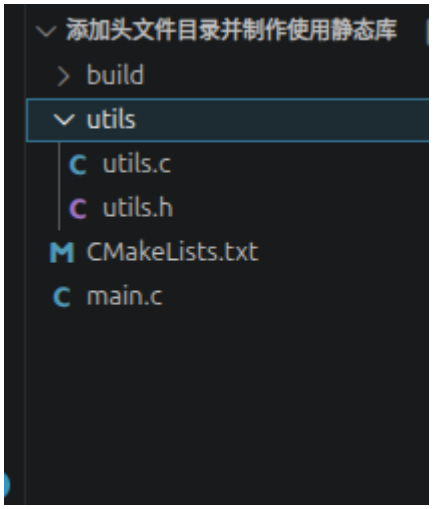


set(SOURCE main.c utils.c)用来将 SOURCE 打包，头文件目录调用使用 target_include_directories(HelloWorld PRIVATE ${CMAKE_CURRENT_SOURCE_DIR}/utils)用来进行库文件的包含，解释为头文件在目前目录的 utils 文件夹下

PRIVATE 表示这个头文件只能被 HelloWorld 自己使用，不传递给依赖它的其他库

制作与使用静态库

将 utils 做成一个库，文件目录格式如下



此时在 utils 文件夹中还需要一个 cmakelists,作为 utils 的库文件构建

顶层 cmakelist

```
cmake_minimum_required(VERSION 3.10)
project(HelloWorld C)
add_subdirectory(utils)
add_executable(HelloWorld main.c)
target_link_libraries(HelloWorld PRIVATE utils)
```

Add_subdirectory 添加 utils 子目录

target_link_libraries 链接 utils 库

子目录 cmakelist

```
add_library(utils STATIC utils.c)
target_include_directories(utils PUBLIC ${CMAKE_CURRENT_SOURCE_DIR})
```

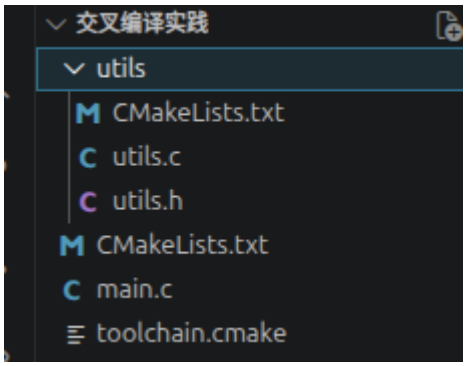
add_libraries 创建 utils 库，static 解释为静态，创建的是 libutils.a 库，源文件为 utils.c

target 给库文件设置路径，PUBLIC 表示使用者也需要这个路径，即 libutils 需要，main 也需要

交叉编译

在 cmakelists 中直接设置使用的工具链

文件结构



对于两个 cmakelist 与原来的写法没有区别，但是多了一个.cmake 文件用于设置编译使用 gcc 交叉编译器，具体如下

toolchain.cmake

```
#输出目标系统为linux
set(CMAKE_SYSTEM_NAME LINUX)
#输出目标系统架构为arm32
set(CMAKE_SYSTEM_PROCESSOR arm)

#设置交叉编译器
set(CMAKE_C_COMPILER arm-buildroot-linux-gnueabihf-gcc)
set(CMAKE_CXX_COMPILER arm-buildroot-linux-gnueabihf-g++)

#查找规则
set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM NEVER)
set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY ONLY)
set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE ONLY)
set(CMAKE_FIND_ROOT_PATH_MODE_PACKAGE ONLY)
```

查找规则的核心作用是 cmake 去搜索文件依赖时去主机找而不是去从机找，.cmake 文件可以通用，在编译的时候需要使用

Cmake .. –DCMAKE_TOOLCHAIN_FILE=../toolchain.cmake

#..指的是去上级目录寻找，因为原来在 build 文件夹中，这个指令的意思是需要使用 toolchain.cmake 这个文件去更改编译器选项

make

实验结果：

```
fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR: ~/桌面/Linux&Makefil...
-- Generating done (0.0s)
CMake Warning:
  Manually-specified variables were not used by the project:

    CMAKE_TOOLCHAIN_FILE


-- Build files have been written to: /home/fanzeny/桌面/Linux&Makefile/交叉编译实践/build
(base) fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR:~/桌面/Linux&Makefile/交叉编译实践/build$ make
[ 25%] Building C object utils/CMakeFiles/utils.dir/utils.c.obj
[ 50%] Linking C static library libutils.a
[ 50%] Built target utils
[ 75%] Building C object CMakeFiles/hello_world.dir/main.c.obj
[100%] Linking C executable hello_world
[100%] Built target hello_world
(base) fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR:~/桌面/Linux&Makefile/交叉编译实践/build$ file hello_world
hello_world: ELF 32-bit LSB pie executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux-armhf.so.3, for GNU/Linux 3.2.0, not stripped
(base) fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR:~/桌面/Linux&Makefile/交叉编译实践/build$
```

文件显示为 arm 架构

shell 脚本学习与使用

本质是把一系列终端指令写进一个文件，一次性执行

基础语法

第一行 #!/bin/bash 运行 bash 终端

输出文字 echo “Hello,World”

变量定义与使用 NAME=“rk3506”(定义时等号两边不能有空格，使用时前边加 $)

```
echo $NAME
```

获取命令与结果 NOW=$(date) ($()可以把命令的输出赋值给变量)

```
条件判断 if [ -f “hello.c”]; then    [ ]中的是测试命令，if 和 fi 中间的命令是 if 成立的运行命令
```

```
echo “存在”                    -f 是文件存在而且是普通文件的意思(不是目录或者链接)，
if
                                                          在返回值为 0 的时候才执
行，这点注意
fi
```

循环 `for file in *.c; do` 遍历所有的.c 文件，放入变量 file，输出 `file` 的值

`Echo $file`

`done`

文件操作指令

创建文件夹 mkdir –p /userdata/my_project -p 指的是父文件目录不存在时自动创建

创建空文件 touch `/userdata/readme.txt`

复制文件 cp /mnt/sdcard/*.ko /userdata/ 在这里空格左侧是复制文件的路径，右侧是粘贴的路

径，这里意思是把目录下所有的.ko 文件复制到 userdata

移动/重命名 mv old.text new.txt 和复制文件一样，空格隔开

删除文件 rm –f build.*.o -f 指的是强制删除不提醒

删除目录 rm –rf /userdata/`build/` 会删掉 build，不会删掉 userdata

查看当前路径 pwd

读取文件内容 cat config.txt CONTENT=$(cat config.txt)

写入文件 echo “VERSION=1.0” > version.h 清空原有的内容再写入

追加写入 echo “VERSION=1.0” >> version.h 追加到文件末尾，新开行

实验：自动化编译并执行 c 程序，电脑本地

我写的.sh 如下

$ build_and_run.sh × M CMakeLists.txt

$ build_and_run.sh

```bash
#!/bin/bash
TARGET="myapp"
BUILD_DIR="build"   # Build directory

echo "Building $TARGET..."
mkdir -p $BUILD_DIR
cd $BUILD_DIR

echo "Compiling source files..." #Cmake
cmake ..
if [ $? -ne 0 ]; then #命令返回值检查， #?是上一个命令的返回值，0表示成功，非0表示失败
    echo "CMake configuration failed!"
    exit 1
fi

make


if [ -f "$TARGET" ]; then
    echo "运行$TARGET..."
    ./$TARGET
else
    echo "Error: $TARGET not found!"
    exit 1
fi

cd .. #返回上一级目录
echo "Done."
```

实验结果如下

```
fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR: ~/桌面/Shell/本地
(base) fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR:~/桌面/Shell/本地$ chmod +x build_and_run
.sh
(base) fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR:~/桌面/Shell/本地$ ./build_and_run.sh
Building myapp...
Compiling source files...
-- The C compiler identification is GNU 13.3.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Configuring done (0.4s)
-- Generating done (0.0s)
-- Build files have been written to: /home/fanzeny/桌面/Shell/本地/build
Error: myapp not found!
(base) fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR:~/桌面/Shell/本地$ ./build_and_run.sh
Building myapp...
Compiling source files...
-- Configuring done (0.0s)
-- Generating done (0.0s)
-- Build files have been written to: /home/fanzeny/桌面/Shell/本地/build
[ 33%] Building C object CMakeFiles/myapp.dir/main.c.o
[ 66%] Building C object CMakeFiles/myapp.dir/utils.c.o
[100%] Linking C executable myapp
[100%] Built target myapp
运行myapp...
Hello, World!
```

交叉编译脚本

```
$ build_and_run.sh
#!/bin/bash
TARGET="myapp"
BUILD_DIR="build"   # Build directory

echo "Building $TARGET..."
mkdir -p $BUILD_DIR
cd $BUILD_DIR

echo "Compiling source files..." #Cmake
cmake .. -DCMAKE_TOOLCHAIN_FILE=../toolchain.cmake
if [ $? -ne 0 ]; then #命令返回值检查，  #?是上一个命令的返回值，0表示成功，非0表示失败
    echo "CMake configuration failed!"
    exit 1
fi

make
#! 表示取反，将真变为假，在[]中是专门用来判断条件的，管道指令不能放在[]中

if ! file "$TARGET" | grep -q "ARM"; then  #grep输出值为0则检索成功，非0则失败，|表示管道，将前一个命令的输出作为后一个命令的输入,不显示在屏幕上，-q表示安静模式，不输出任何内容
    echo "Error: $TARGET is not an ARM executable!"
    exit 1
else
    echo "Successfully built $TARGET."
fi

cd .. #返回上一级目录
echo "Done."
```

实验结果

fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR: ~/桌面/Shell/交叉编译

```
le on discourse.cmake.org.
-- Configuring done (0.0s)
-- Generating done (0.0s)
-- Build files have been written to: /home/fanzeny/桌面/Shell/交叉编译/build
[100%] Built target myapp
./build_and_run.sh: 第 19 行:  [: 缺少 "]"
Successfully built myapp.
Done.
(base) fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR:~/桌面/Shell/交叉编译$ ./buil
d_and_run.sh
Building myapp...
Compiling source files...
System is unknown to cmake, create:
Platform/LINUX to use this system, please post your config file on discourse.cma
ke.org so it can be added to cmake
Your CMakeCache.txt file was copied to CopyOfCMakeCache.txt. Please post that fi
le on discourse.cmake.org.
-- Configuring done (0.0s)
-- Generating done (0.0s)
-- Build files have been written to: /home/fanzeny/桌面/Shell/交叉编译/build
[100%] Built target myapp
Successfully built myapp.
Done.
(base) fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR:~/桌面/Shell/交叉编译$
```